

SOFTWARE ENGINEERING (SE) PRACTICAL & VIVA MASTER GUIDE

Comprehensive Preparation for Hospital Management System (HMS)

Team: Anish Chitnis (24CE1044) | Aaryan Choube (24CE1045) | Somansh Dafade (24CE1047)

Batch: A / A1 | RAIT, DY Patil University

Software Engineering (SE) Master Viva & Practical Guide

Project: Hospital Management System (HMS)

Team Members: Anish Chitnis (24CE1044), Aaryan Choube (24CE1045), Somansh Dafade (24CE1047)

Batch: A / A1 | **Department:** Computer Engineering, RAIT, DY Patil University

SECTION 1: HMS PROJECT DOMAIN & ARCHITECTURE OVERVIEW

Q1: What is the Hospital Management System (HMS)?

Ans: HMS is an integrated, web-based healthcare enterprise platform designed to automate clinical, diagnostic, pharmaceutical, and financial workflows in a multi-specialty hospital. It replaces paper ledgers with a centralized relational database and role-based access for patients, doctors, receptionists, pharmacists, lab technicians, and hospital administrators.

Q2: What is the System Architecture of HMS?

Ans: HMS follows a **3-Tier Client-Server Architecture**:

- Presentation Layer (Frontend):** Responsive Web UI built using HTML5/React/CSS, enabling role-specific dashboards.
- Application / Business Logic Layer (Backend):** RESTful Web Services (Spring Boot / Node.js) implementing business rules, appointment scheduling, and automated billing logic (calculateBill).
- Data Storage Layer (Database):** Relational Database (MySQL / PostgreSQL) storing 7 core data stores (Patient Master, Schedules, EHR, Lab Reports, Pharmacy Stock, Billing Ledgers, Audit Logs).

Q3: Who are the Primary Actors / Stakeholders?

- Patient:** Registers, views doctor slots, books appointments, accesses digital prescriptions and lab reports, pays bills online.
- Receptionist:** Registers walk-in patients, issues queue tokens, verifies doctor availability, manages bed/ward admissions.
- Doctor:** Reviews patient history, records clinical diagnoses, issues electronic prescriptions, orders laboratory tests.
- Pharmacist:** Verifies digital prescriptions, dispenses medicines, updates drug inventory, monitors low-stock alerts.

- **Laboratory Technician:** Receives test requisitions, collects specimen samples, uploads diagnostic test results.
- **Hospital Administrator:** Configures staff accounts, manages role-based access control (RBAC), views hospital occupancy and revenue MIS reports.
- **Payment Gateway (External Entity):** Authorizes digital payments (UPI, Cards, Net Banking) and returns transaction status tokens.

📦 SECTION 2: PROCESS MODEL & PROBLEM STATEMENT (EXPERIMENT 1 & 2)

Q4: What problems existed in the manual hospital system?

1. **Long Queues & Scheduling Chaos:** Manual token counters and appointment overlapping.
2. **Fragmented & Misplaced Medical Charts:** Paper records lost or delayed during cross-department emergencies.
3. **Pharmacy Inventory Mismatches:** Unrecorded dispensing leading to stockouts of life-saving medicines and expired batch waste.
4. **Billing Inaccuracies & Revenue Leakage:** Disconnect between lab tests, pharmacy, and cashier desks leading to under-billing or delayed discharge.
5. **No Auditability:** Lack of security logs for sensitive patient health data.

Q5: Why did you choose the V-Model instead of Waterfall or pure Agile?

Ans (Crucial Viva Justification):

1. **Well-Defined, Stable Requirements:** Healthcare administration workflows follow standardized institutional protocols frozen upfront in the IEEE 830 SRS document.
2. **Correctness-Critical Healthcare Domain:** Medical errors or billing glitches carry legal liability and patient health risks. The V-Model enforces **parallel verification and validation planning**:
 - **Requirements Phase** → Plans **Acceptance Testing**
 - **System Architecture Phase** → Plans **System Testing**
 - **High-Level Design Phase** → Plans **Integration Testing**
 - **Module Design Phase** → Plans **Unit / Basis Path Testing**
3. **1-to-1 Traceability:** Every functional requirement (FR-1 to FR-6) maps directly to a verification test case.
4. **Semester Academic Timeline:** Fixed milestone reviews align naturally with the deterministic phases of the V-Model.

Q6: What is the IEEE 830 Standard for SRS?

Ans: IEEE 830 specifies the structure of a Software Requirements Specification:

1. Introduction (Purpose, Scope, Definitions, References)
2. Overall Description (Product perspective, user classes, operating environment, design constraints)
3. Specific Requirements (External interfaces, Functional Requirements FR-x, Non-Functional Requirements: Security, HIPAA Compliance, Performance, Reliability)

SECTION 3: DATA FLOW DIAGRAMS & DATA DICTIONARY (EXPERIMENT 3)

Q7: What are the DFD Notations used?

- **External Entity:** Rectangle (Source/sink outside system: Patient, Doctor, Gateway).
- **Process:** Circle / Rounded Rectangle (Transforms data: `1.0 Patient Registration`).
- **Data Store:** Open rectangle / Parallel lines (Holds data: `D1 Patient Master`).
- **Data Flow:** Directed arrow (Carries data packets).

Q8: What are the strict Rules of DFD?

1. **No Entity-to-Entity direct flow:** Data must be processed by a system process.
2. **No Entity-to-Store direct flow:** Cannot read/write a database directly without a process.
3. **No Store-to-Store direct flow:** Data movement requires a computational process.
4. **Conservation of Data:** A process cannot produce output without necessary input data (no "miracles" or "black holes").

Q9: Explain Level 0, Level 1, and Level 2 DFD of HMS.

- **Level 0 (Context Diagram):** System represented as single process `0: Hospital Management System`, interacting with 6 external entities.
- **Level 1 DFD:** Decomposed into 6 core processes:
 - `1.0` Patient Registration & Appointment
 - `2.0` Doctor Consultation & EHR
 - `3.0` Laboratory & Diagnostics
 - `4.0` Pharmacy & Stock Inventory
 - `5.0` Billing & Payment Processing
 - `6.0` Hospital Administration & Reporting
- Along with 7 Data Stores (`D1` to `D7`).
- **Level 2 DFD:** Expands Process `5.0 (Billing & Payment)` into 4 sub-processes:
 - `5.1` Aggregate Service Charges (from D3 EHR, D4 Lab, D5 Pharmacy)
 - `5.2` Verify Insurance & Apply Discounts
 - `5.3` Process Payment Transaction (Counter cash or Payment Gateway)
 - `5.4` Generate Itemized Invoice & Update Ledger (writes D6)

Q10: What is a Data Dictionary?

Ans: A repository containing metadata about all data elements, composite flows, and stores. Uses BNF notation:

- `+` = AND (Concatenation)
- `[ | ]` = OR (Selection)
- `{}` = Iteration (Zero or more occurrences)
- `()` = Optional data element

SECTION 4: PROJECT SCHEDULING (GANTT CHART & PERT/CPM) (EXPERIMENT 4)

Q11: What is the difference between PERT and CPM?

- **CPM (Critical Path Method):** Deterministic technique used when task durations are known with certainty (e.g., standard software development). Focuses on cost-time trade-offs.
- **PERT (Program Evaluation and Review Technique):** Probabilistic technique used for research/R&D projects where durations are uncertain. Uses 3 time estimates:

$$T_e = \frac{a + 4m + b}{6}$$

where \$a\$ = optimistic time, \$m\$ = most likely time, \$b\$ = pessimistic time.

Q12: What is the Critical Path and Total Project Duration for HMS?

- **Total Duration:** 47 Working Days.
- **Critical Path:** A → B → C → D → F → I → J → K
- A: Requirement Analysis (5d)
- B: SRS IEEE 830 Preparation (4d)
- C: System Design DFD/UML (6d)
- D: Database Design & Schema (5d)
- F: Patient Registration & Appointment Module (9d)
- I: Billing & Payment Gateway Integration (9d)
- J: Integration & White-Box Testing (6d)
- K: Deployment & User Training (3d)
- **Total Float on Critical Path:** 0 Days (Zero Slack). Any delay on these tasks directly delays the entire 47-day delivery.

SECTION 5: AGILE SCRUM & JIRA PROJECT MANAGEMENT (EXPERIMENT 5)

Q13: What are the key Scrum Roles and Events?

- **Roles:**
- **Product Owner (PO):** Defines user stories, maintains and prioritizes Product Backlog.
- **Scrum Master (SM):** Serves the team, removes impediments, ensures Scrum adherence.
- **Development Team:** Cross-functional professionals who build and test increments.
- **Events (Ceremonies):**
- **Sprint Planning:** Select stories from Product Backlog to form the Sprint Backlog.
- **Daily Scrum (Standup):** 15-minute daily sync (Yesterday, Today, Blockers).
- **Sprint Review:** Demo of working software increment to stakeholders.
- **Sprint Retrospective:** Team inspects its own processes to improve next sprint.

Q14: How many Sprints did HMS have?

Ans: 5 Sprints (each ~2 weeks / 9–10 days):

- Sprint 1: Setup, Auth, RBAC & Patient Registration (18 story points)
- Sprint 2: Doctor Schedule & Appointment Booking (21 story points)
- Sprint 3: Doctor EHR, Clinical Notes & Diagnostics (19 story points)
- Sprint 4: Automated Billing, Insurance Discount & Payment Gateway (23 story points)
- Sprint 5: White-Box Testing, Security Audits & Deployment (15 story points)

SECTION 6: UML MODELING (USE CASE, CLASS, ACTIVITY, SEQUENCE, COLLABORATION)

Q15: What is the difference between `<<include>>` and `<<extend>>`?

- **`<<include>>` (Mandatory Dependency):** The base use case *cannot* complete without executing the included use case.
- **Example in HMS:** `Book Appointment` **`<<include>>`** `Verify Doctor Availability`.
- **Example in HMS:** `Calculate Total Bill` **`<<include>>`** `Process Payment Transaction`.
- **`<<extend>>` (Optional / Conditional Dependency):** The extending use case executes only under specific conditions (extension points).
- **Example in HMS:** `Cancel / Reschedule Appointment` **`<<extend>>`** `Book Appointment` (triggered only if patient desires change).
- **Example in HMS:** `Apply Health Insurance Discount` **`<<extend>>`** `Calculate Total Bill` (triggered only for insured patients).

Q16: Explain Aggregation vs. Composition in the HMS Class Diagram.

- **Aggregation (Hollow Diamond `o--`):** Weak "has-a" ownership. If the whole is destroyed, the parts still exist.
- **HMS Example:** `Department o-- Doctor`. A Department has Doctors. If a Department is closed or reorganized, the Doctor objects still exist in the hospital.
- **Composition (Filled Diamond `*--`):** Strong "contains" lifecycle ownership. If the whole is destroyed, the parts are destroyed with it.
- **HMS Example:** `MedicalRecord *-- Prescription`. A Prescription cannot exist independently without a parent Medical Record.
- **HMS Example:** `Bill *-- BillItem`. Individual itemized charge lines have no meaning without the parent Bill.

Q17: What are Swimlanes, Fork, and Join in the Activity Diagram?

- **Swimlanes:** Partitions showing actor responsibility (Patient, Receptionist, Doctor, Pharmacy/Lab, Billing System).
- **Fork (Thick Bar):** Splits single control flow into two or more **concurrent/parallel** flows.
- **HMS Example:** After Doctor diagnosis, the flow forks into **Branch 1** (Conduct Diagnostic Lab Test) and **Branch 2** (Verify & Dispense Medicines) **simultaneously**.
- **Join (Thick Bar):** Synchronizes multiple parallel flows back into a single thread. Billing execution waits for both lab reports and pharmacy dispensation to complete before computing the final bill.

Q18: Compare Sequence Diagram vs Collaboration Diagram.

Feature	Sequence Diagram	Collaboration (Communication) Diagram
Primary Focus	**Time sequence** of interactions	**Structural links** between objects
Lifelines	Explicit vertical dashed lifelines	No lifelines (objects represented as rectangles)
Message Ordering	Implicit (top-to-bottom time axis)	Explicit **numbered messages** (`1:`, `2:`, `3:`)
Usage	Best for tracing operational execution	Best for evaluating impact of structural refactoring

SECTION 7: COHESION AND COUPLING (EXPERIMENT 9)

Q19: What is Cohesion? What are its types from Best to Worst?

****Ans:**** Cohesion measures the degree to which elements within a single module belong together (Intra-module strength). High cohesion is desired.

1. ****Functional Cohesion (BEST):**** Module performs exactly one single, well-defined mathematical/business task (e.g., `Patient Registration`, `calculateBill`).
2. ****Sequential Cohesion:**** Output of one statement is input to the next (e.g., Aggregating bill components then calculating tax).
3. ****Communicational Cohesion:**** Operates on the same input data set.
4. ****Procedural Cohesion:**** Order of execution matters.
5. ****Temporal Cohesion:**** Operations executed at the same time (e.g., system startup initialization).
6. ****Logical Cohesion:**** Related by category (e.g., all input routines in one class).
7. ****Coincidental Cohesion (WORST):**** Arbitrary functions bundled together without meaningful relation.

Q20: What is Coupling? What are its types from Best to Worst?

****Ans:**** Coupling measures the degree of interdependence between two software modules (Inter-module connectivity). Low/loose coupling is desired.

1. ****Data Coupling (BEST):**** Modules communicate solely by passing atomic data parameters (e.g., `calculateBill(fee, labCost, hasIns, isEmerg)`).
2. ****Stamp Coupling:**** Passing composite data structures when only part of the data is needed.
3. ****Control Coupling:**** One module passes flags/parameters controlling the internal execution logic of another.
4. ****External Coupling:**** Modules depend on external formatting/protocols.
5. ****Common Coupling:**** Modules share global data/variables.
6. ****Content Coupling (WORST):**** One module directly modifies or accesses the internal data of another.

SECTION 8: RMMM PLAN (RISK MANAGEMENT) (EXPERIMENT 10)

Q21: What is the full form of RMMM and Risk Exposure formula?

- **RMM: Risk Mitigation, Monitoring, and Management.**
- **Risk Exposure:**

$$\text{Risk Exposure (RE)} = \text{Probability (P)} \times \text{Impact (I)}$$

Q22: What are the 3 pillars of RMMM?

1. **Risk Mitigation (Proactive):** Steps taken before risk occurs to minimize probability (e.g., automated daily backups, role-based access).
2. **Risk Monitoring:** Tracking risk indicators continuously during development (e.g., gateway API error logs, sprint burndown slippage).
3. **Risk Management / Contingency (Reactive):** Action plan executed when the risk actually manifests (e.g., fallback to manual counter receipt entry if payment gateway crashes).

🔖 SECTION 9: SCM & GIT VERSION CONTROL (EXPERIMENT 11)

Q23: What is a Configuration Item (CI) and Baseline?

- **Configuration Item (CI):** Any work product formally placed under version control (SRS, DFD/UML diagrams, DDL database schemas, source code, test cases).
- **Baseline:** A formally reviewed and approved version of a CI that serves as the basis for further development, modified only through formal change control.

Q24: What is the Change Control Board (CCB) process?

1. Stakeholder submits a **Change Request (CR)**.
2. Technical team conducts **Impact Analysis** (cost, schedule, architectural regression).
3. **CCB (Change Control Board)** approves, rejects, or defers the CR.
4. If approved, an engineer checks out the code, implements changes on a branch, runs regression tests, and tags a new baseline.

🔧 SECTION 10: WHITE-BOX TESTING & CYCLOMATIC COMPLEXITY (EXPERIMENT 12)

Q25: Explain the method under test: `calculateBill()`

[illegible]

}

Q26: What is McCabe's Cyclomatic Complexity? Show calculation for HMS.

Ans. Cyclomatic Complexity $V(G)$ defines the upper bound on the number of linearly independent paths through program logic:

- **Graph Metrics:**
 - Nodes $N = 7$
 - Edges $E = 8$
 - Connected Components $P = 1$
 - Predicate Nodes $P_n = 2$ (Node 1 and Node 4)
 - Closed Regions $R = 2$
 - **3 Verification Methods:**
1. **Method 1:** $V(G) = E - N + 2P = 8 - 7 + 2(1) = \mathbf{3}$
 2. **Method 2:** $V(G) = P_n + 1 = 2 + 1 = \mathbf{3}$
 3. **Method 3:** $V(G) = \text{ext}\{\text{Closed Regions}\} + 1 = 2 + 1 = \mathbf{3}$

Q27: What are the 3 Independent Basis Paths and their Test Cases?

1. **Basis Path 1:** \$1

→ 2

- **Inputs:** fee = 0.0, labCost = 1000.0, hasIns = false, isEmerg = false
 - **Expected Output:** -1.0 (Invalid input error) | **Actual:** -1.0 | **Status:** PASSED [OK]
2. **Basis Path 2:** \$1

→ 3

→ 4

→ 5

→ 6

→ 7

- **Inputs:** fee = 2000.0, labCost = 3000.0, hasIns = true, isEmerg = false
 - **Expected Output:** 4500.0 (10% discount on 5000) | **Actual:** 4500.0 | **Status:** PASSED [OK]
3. **Basis Path 3:** \$1

→ 3

→ 4

→ 6

→ 7

- **Inputs:** fee = 1500.0, labCost = 500.0, hasIns = false, isEmerg = false
- **Expected Output:** 2000.0 (Gross bill, no discount) | **Actual:** 2000.0 | **Status:** PASSED [OK]
- **Coverage:** 100% Statement Coverage, 100% Branch Coverage, and 100% Basis Path Coverage.